

Report on Cyber Security Tools

**Created By
Vrushali Walve - 2071**

1. Password Strength Analyzer

Introduction :

The Password Security Analyzer is a cybersecurity utility web application developed using modern web technologies. The main purpose of this project is to help users evaluate the strength of their passwords and check whether their passwords appear in known leaked datasets, without storing any user data.

This tool follows ethical security practices by ensuring that passwords are never saved in databases or files. Instead, passwords are processed temporarily in memory and discarded after analysis.

Objectives of the Project :

- To analyze password strength using industry-standard algorithms
- To check passwords against a local leaked dataset
- To ensure zero password storage for security and privacy
- To create a user-friendly cybersecurity utility

Technologies Used :

Frontend:

- React.js
- Tailwind CSS
- Vite

Backend:

- Node.js
- Express.js

Security Libraries:

- zxcvbn (password strength analysis)
- Crypto module (hashing)

System Architecture :

The system follows a simple client-server architecture:

Frontend (React) → Backend (Node + Express) → Local leaked dataset file

The frontend sends the password to the backend API. The backend hashes the password and compares it with stored leaked password hashes. The result is returned to the frontend without storing any sensitive data.

Features :

- Password input with show/hide option
- Real-time password strength scoring
- Visual strength meter
- Leak detection using local dataset
- Security feedback and tips
- No database usage for password storage

Advantages :

- Ethical handling of user data
- Fast performance due to local dataset
- Easy to use interface
- No risk of password leaks
- Suitable for security awareness

Limitations :

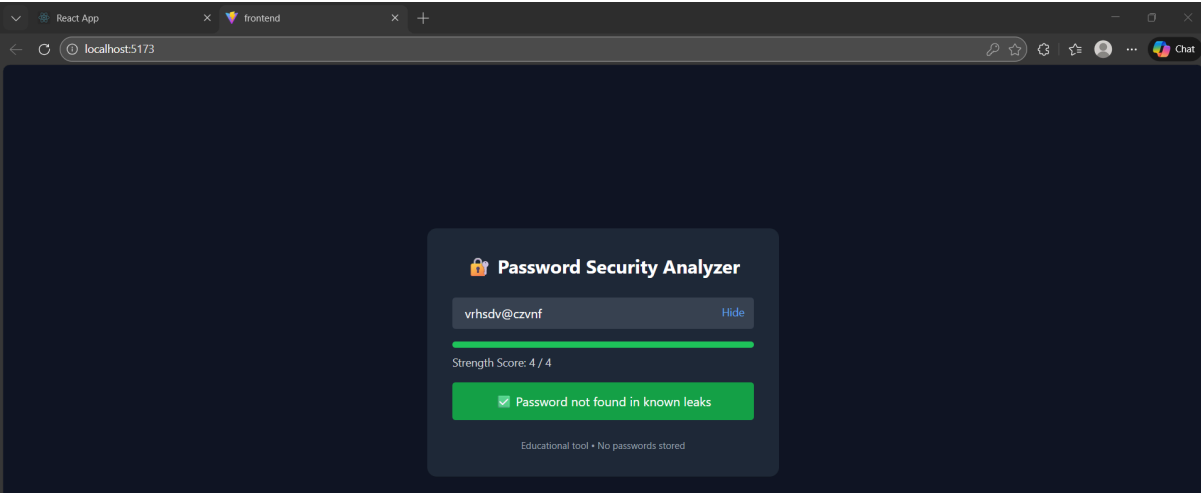
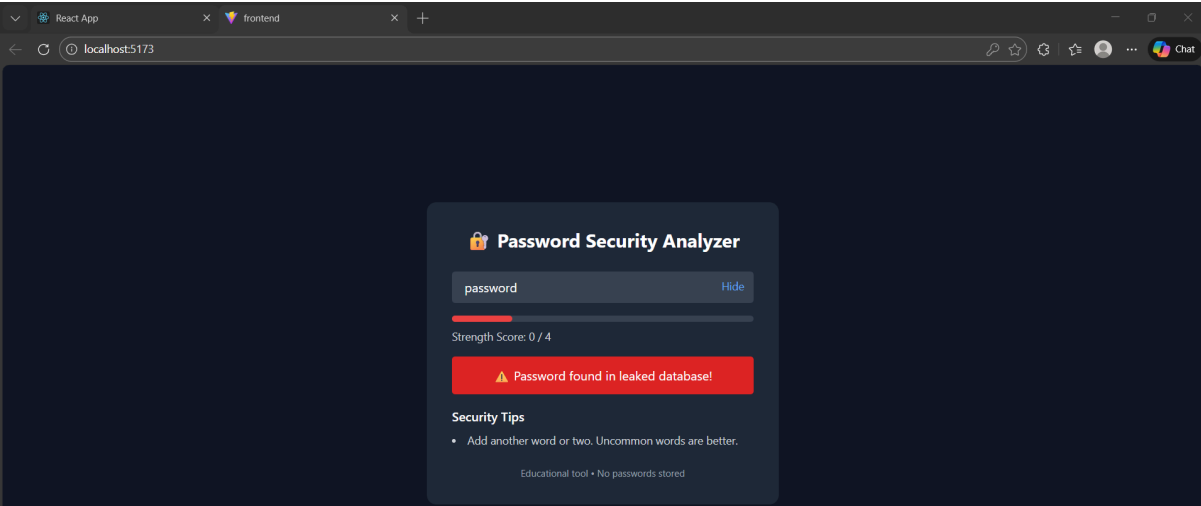
- Uses a limited simulated leaked dataset
- Does not support real-time online breach databases
- No user history tracking

Conclusion :

The Password Security Analyzer successfully demonstrates a secure and ethical approach to password analysis. By combining modern frontend technologies with secure backend practices, the project delivers a practical cybersecurity utility without compromising user privacy.

This project reflects real-world security principles and provides a strong foundation for future enhancements.

PUC :



2. Secure File Upload + Malware Scan Portal

Introduction :

With the increasing use of digital file sharing, malware attacks through infected files have become a major cybersecurity threat. Many users unknowingly upload malicious files that can compromise systems and networks.

The Secure File Upload & Malware Scan Portal is a web-based application developed using the MERN Stack (MongoDB, Express.js, React.js, Node.js) that allows users to upload files securely and scan them for malware. The system generates scan results, maintains a scan history, and provides downloadable PDF reports for each scan.

This project aims to demonstrate secure file handling, backend processing, database storage, and cybersecurity concepts in a user-friendly interface.

Objectives :

- To provide a secure platform for uploading files
- To scan uploaded files for malware threats
- To generate SHA256 hash for file integrity verification
- To store scan history in a database
- To allow users to download scan reports in PDF format
- To create a simple and professional cybersecurity dashboard

Technologies Used :

Frontend:

- React.js
- HTML (JSX)
- CSS (Dark themed UI)

Backend:

- Node.js
- Express.js

Database:

- MongoDB (NoSQL)

Security Tools:

- ClamAV (antivirus scanning logic – simulated in development)

System Architecture :

The system follows a simple client-server architecture:

Frontend (React) → Backend (Node.js + Express) → Malware Scan Engine (ClamAV Logic) → MongoDB Database

The frontend provides a user interface where users can upload files for scanning. Once a file is selected, it is sent securely to the backend through REST APIs. The backend handles file uploads using Multer middleware, generates a SHA256 hash for file integrity verification, and processes the file through the malware scanning logic.

After scanning, the backend stores the scan details such as filename, hash value, scan result, and timestamp in the MongoDB database. The scan result is then returned to the frontend and displayed to the user. The frontend also retrieves scan history from the backend and allows users to download scan reports in PDF format.

This architecture ensures separation of concerns between the user interface, application logic, security processing, and data storage.

Features :

- Secure file upload system
- Malware scanning functionality
- SHA256 hash generation
- Scan history dashboard
- PDF scan report download
- Delete single scan record
- Clear complete scan history
- Dark-themed cybersecurity UI

Advantages :

- Easy-to-use interface
- Quick malware detection process
- Organized scan history
- Downloadable security reports
- Demonstrates real-world cybersecurity workflow
- MERN Stack full implementation

Limitations :

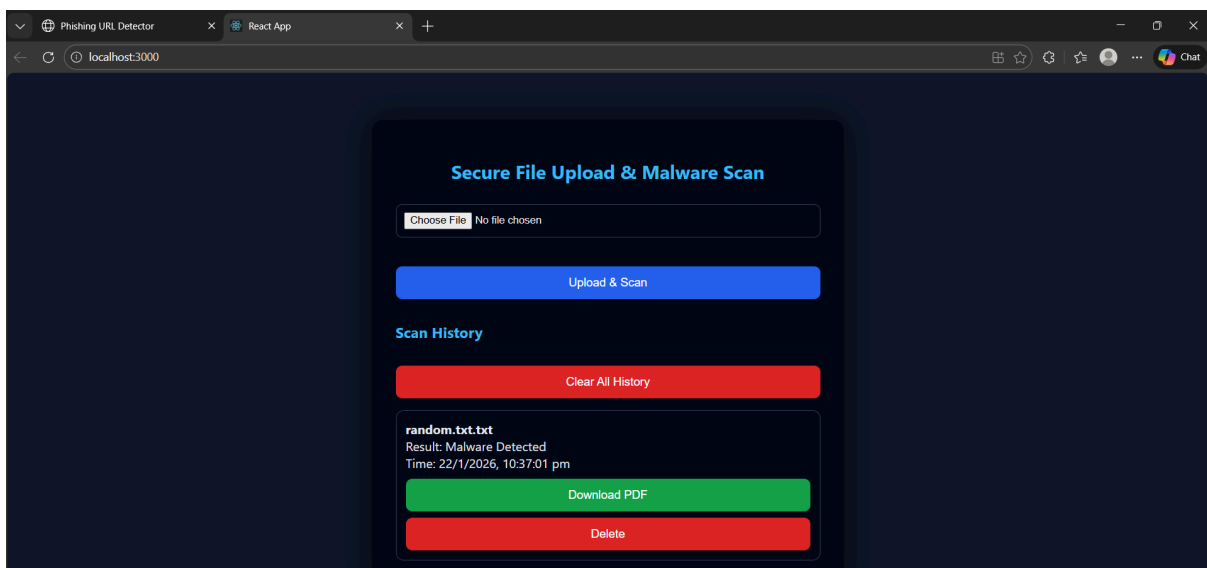
- Real-time ClamAV scanning not enabled on Windows environment
- No user authentication in current version
- Suitable mainly for small-scale usage

Conclusion :

The Secure File Upload & Malware Scan Portal successfully demonstrates the integration of cybersecurity concepts with full-stack web development using the MERN Stack. The system provides a secure and efficient way to upload files, scan them for malware, and maintain scan records with professional reporting features.

This project helps in understanding real-world backend security processing, database handling, and frontend interaction while focusing on cybersecurity utilities. It serves as a strong foundation for further enhancements and real enterprise-level security applications.

PUC :



3. Phishing URL Detector

Introduction :

The Phishing URL Detector is a cybersecurity utility web application designed to identify whether a given URL is phishing or legitimate. The primary goal of this project is to help users avoid malicious websites by analyzing URL patterns and predicting risks using machine learning techniques along with heuristic rules.

The tool follows secure practices by not storing any scanned URLs permanently. Each URL is processed temporarily to extract features and generate predictions, ensuring user privacy and data safety.

Objectives :

- To detect phishing websites using machine learning algorithms
- To analyze URL characteristics and suspicious patterns
- To provide risk-based classification for better accuracy
- To develop a simple and interactive cybersecurity web tool
- To demonstrate integration of ML with web technologies

Technologies Used :

Backend & ML:

- Flask (web framework)
- Pandas (data processing)
- Scikit-learn (machine learning)
- Joblib (model storage)

Frontend:

- Embedded HTML and CSS within Flask

System Architecture :

The system follows a client-server architecture combined with a machine learning pipeline:

User Interface (Flask Web App) → Feature Extraction Module → ML Prediction Model → Risk Scoring Logic → Result Display

The user enters a URL into the web interface. The backend extracts lexical features from the URL, applies the trained machine learning model, calculates a risk score based on heuristic rules, and displays the final result.

Features :

- URL input interface
- Machine learning-based phishing detection
- Risk percentage scoring
- Hybrid detection approach (ML + rules)
- Fast real-time prediction
- No storage of scanned URLs

Advantages :

- Helps prevent phishing attacks
- Quick and accurate detection
- User-friendly interface
- No privacy risks

Limitations :

- Model depends on dataset quality
- Limited handcrafted features
- Does not analyze website content
- Offline detection only

Conclusion :

The Phishing URL Detector successfully demonstrates an intelligent cybersecurity tool combining machine learning and heuristic analysis. It provides users with real-time phishing detection while maintaining data privacy.

The project showcases practical implementation of security concepts and highlights the importance of proactive phishing prevention techniques.

PUC :

